Hugging Face

What is Hugging Face

Hugging Face is a company and open-source community that focuses on Natural Language Processing (NLP) and Artificial Intelligence (AI). It is best known for its Transformers library, which provides tools and pre-trained models for a wide range of NLP tasks, such as text classification, sentiment analysis, machine translation, and more.

Features

- **Transformers Library:** A comprehensive library that includes thousands of pre-trained models like BERT, GPT, T5, and others, which can be fine-tuned for specific tasks.
- **Model Hub:** A platform where users can share and download pre-trained models, datasets, and other resources.
- **Datasets Library:** Provides easy access to various datasets for NLP tasks.
- **Spaces:** A platform for hosting and sharing machine learning demos and applications.
- **Inference API:** Allows users to deploy and use models in production environments easily.

Popular Models On hugging Face

- **BERT:** Bidirectional Encoder Representations from Transformers, used for understanding the context of words in a sentence.
- **GPT:** Generative Pre-trained Transformer, used for text generation tasks.
- **T5:** Text-To-Text Transfer Transformer, which treats every NLP task as a text-to-text problem.
- **RoBERTa:** A robustly optimized BERT approach, offering improved performance on various NLP tasks.

Pipeline object

- The most basic object in the 🤗 Transformers library is the pipeline() function. It connects a model with its necessary preprocessing and postprocessing steps, allowing us to directly input any text and get an intelligible answer:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
classifier("I've been waiting for a HuggingFace course my whole life.")
```

```
[{'label': 'POSITIVE', 'score': 0.9598047137260437}]
```

Pipeline object

- By default, this pipeline selects a particular pretrained model that has been fine-tuned for sentiment analysis in English. The model is downloaded and cached when you create the classifier object.
- If you rerun the command, the cached model will be used instead and there is no need to download the model again.
- There are three main steps involved when you pass some text to a pipeline:

  The text is preprocessed into a format the model can understand.

  The preprocessed inputs are passed to the model.

  The predictions of the model are post-processed, so you can make sense of them.

Available Pipelines

- The pipeline() function supports multiple modalities,

  Text pipelines =>

  text-generation: Generate text from a prompt

  text-classification: Classify text into predefined categories

  summarization: Create a shorter version of a text while preserving key information

  translation: Translate text from one language to another

  zero-shot-classification: Classify text without prior training on specific labels

  feature-extraction: Extract vector representations of text

Available pipelines

Image pipelines

image-to-text: Generate text descriptions of images

image-classification: Identify objects in an image

object-detection: Locate and identify objects in images

Audio pipelines

automatic-speech-recognition: Convert speech to text

audio-classification: Classify audio into categories

text-to-speech: Convert text to spoken audio

Multimodal pipelines

image-text-to-text: Respond to an image based on a text prompt

Zero-Shot Classification

- There are more challenging task where we need to classify texts that haven’t been labelled.
- This is a common scenario in real-world projects because annotating text is usually time-consuming and requires domain expertise. For this use case, the zero-shot-classification pipeline is very powerful
- it allows you to specify which labels to use for the classification, so you don’t have to rely on the labels of the pretrained model.
- This pipeline is called zero-shot because you don’t need to fine-tune the model on your data to use it. It can directly return probability scores for any list of labels you want!

Zero-Shot Classification

```
from transformers import pipeline

classifier = pipeline("zero-shot-classification")
classifier(
    "This is a course about the Transformers library",
    candidate_labels=["education", "politics", "business"],
)
```

Text Generation

- The main idea here is that you provide a prompt and the model will auto-complete it by generating the remaining text.
- This is similar to the predictive text feature that is found on many phones. Text generation involves randomness, so it’s normal if you don’t get the same results

Text Generation

```python
from transformers import pipeline

generator = pipeline("text-generation")
generator("In this course, we will teach you how to")
```

```
[{'generated_text': 'In this course, we will teach you how to understand and use '
                    'data flow and data interchange when handling user data. We '
                    'will be working with one or more of the most commonly used '
                    'data flows — data flows of various types, as seen by the '
                    'HTTP'}]
```

model from the Hub in a pipeline

- you can also choose a particular model from the Hub to use in a pipeline for a specific task.
- text generation. Go to the Model Hub and click on the corresponding tag on the left to display only the supported models for that task.
- You can refine your search for a model by clicking on the language tags, and pick a model that will generate text in another language. The Model Hub even contains checkpoints for multilingual models that support several languages.
- Once you select a model by clicking on it, you'll see that there is a widget enabling you to try it directly online. This way you can quickly test the model's capabilities before downloading it.

model from the Hub in a pipeline

```
from transformers import pipeline

generator = pipeline("text-generation", model="HuggingFaceTB/SmolLM2-360M")
generator(
    "In this course, we will teach you how to",
    max_length=30,
    num_return_sequences=2,
)
```

Mask filling

- The idea of this task is to fill in the blanks in a given text
- The top_k argument controls how many possibilities you want to be displayed. Note that here the model fills in the special <mask> word, which is often referred to as a mask token.
- Other mask-filling models might have different mask tokens, so it’s always good to verify the proper mask word when exploring other models. One way to check it is by looking at the mask word used in the widget.

Mask filling

```python
from transformers import pipeline

unmasker = pipeline("fill-mask")
unmasker("This course will teach you all about <mask> models.", top_k=2)
```

```
[{'sequence': 'This course will teach you all about mathematical models.',
  'score': 0.19619831442832947,
  'token': 30412,
  'token_str': ' mathematical'},
 {'sequence': 'This course will teach you all about computational models.',
  'score': 0.04052725434303284,
  'token': 38163,
  'token_str': ' computational'}]
```

Beyond the Pipeline: The Core Components

- The pipeline is great, but it hides two crucial components that you need to control for fine-tuning: the Tokenizer and the Model.
- Also to get the details of the underlying things we need to use these components.

The Tokenizer

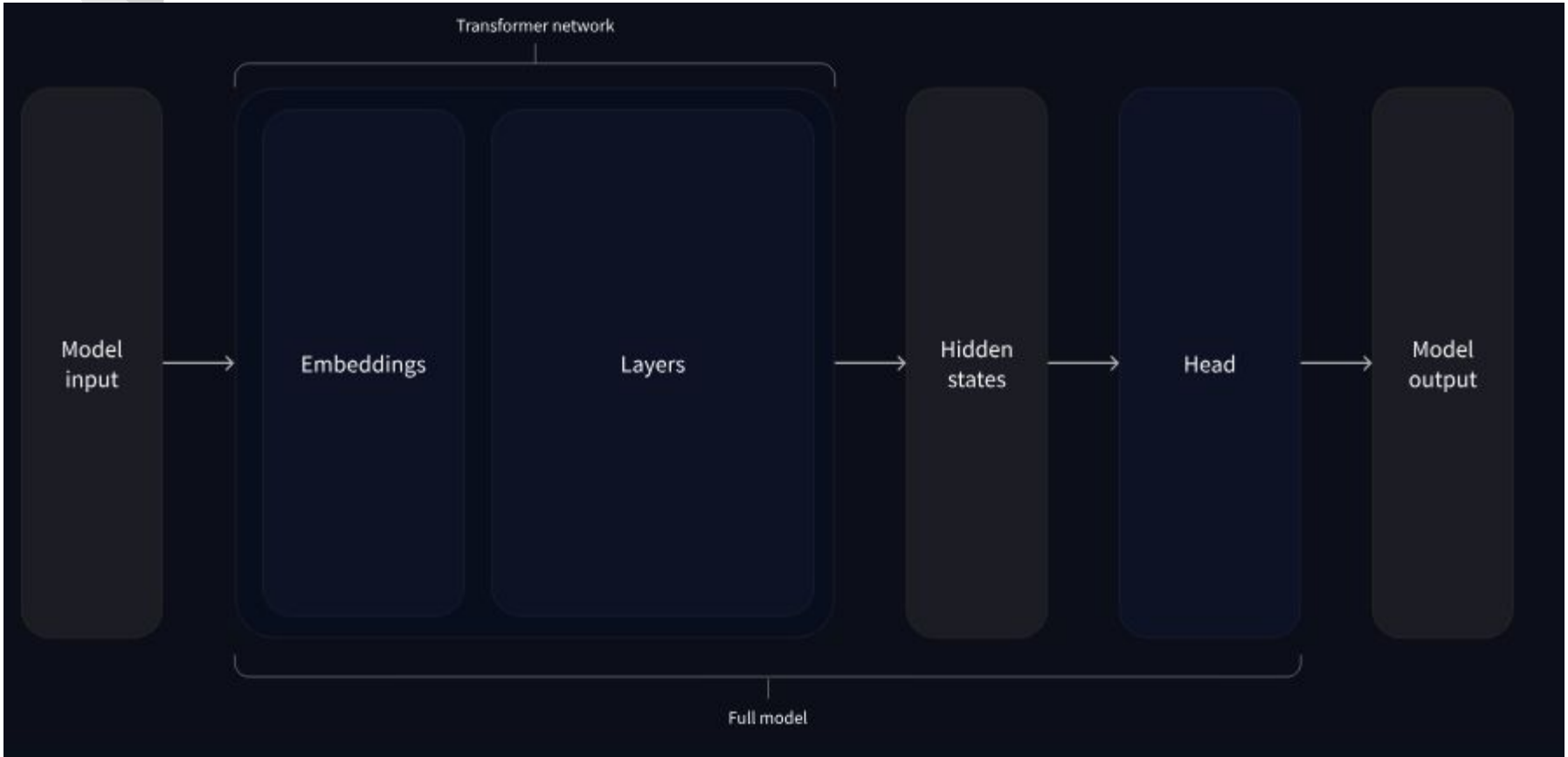
- A tokenizer's job is to translate human-readable text into numbers that a model can understand. It's more than just splitting words.
- What it does:
  - Tokenization: Splits text into "tokens," which are often words, sub-words, or characters. For example, "unbelievably" might become ["un", "##believab", "##ly"].
  - ID Conversion: Converts these tokens into unique integer IDs from a fixed vocabulary.
  - Special Tokens: Adds model-specific special tokens like [CLS] (start of a sequence) and [SEP] (separator).
  - Attention Mask: Creates a binary mask ([1, 1, ..., 0, 0]) to tell the model which tokens are real and which are just padding added to make sequences the same length.

The Tokenizer

- A tokenizer's job is to translate human-readable text into numbers that a model can understand. It's more than just splitting words.
- What it does:
  - Tokenization: Splits text into "tokens," which are often words, sub-words, or characters. For example, "unbelievably" might become ["un", "##believab", "##ly"].
  - ID Conversion: Converts these tokens into unique integer IDs from a fixed vocabulary.
  - Special Tokens: Adds model-specific special tokens like [CLS] (start of a sequence) and [SEP] (separator).
  - Attention Mask: Creates a binary mask ([1, 1, ..., 0, 0]) to tell the model which tokens are real and which are just padding added to make sequences the same length.

The Model

- The model is the neural network itself—the "brain" that processes the tokenized inputs and makes a prediction.
- When you call a model with input_ids and an attention_mask, it passes them through its many layers and outputs raw prediction scores called logits. Logits are not probabilities; they are the final raw scores for each possible class.
- Logits are the raw, unnormalized scores that a model produces right before its final prediction.
- So if they aren't probabilities, why use them? Because they are the perfect input for a function called Softmax. The Softmax function is what takes these raw, unbounded logits and elegantly converts them into probabilities that are all between 0 and 1 and sum up to 1.


Transformer network
Model input
Embeddings
Layers
Hidden states
Head
Model output
Full model

Automodel vs AutoModelForSequenceClassification

- The difference is all about the "head" of the model.
- **AutoModel (The Engine)**
  - Think of AutoModel as just the core engine of a car. It loads a base transformer model (like BERT, DistilBERT, or RoBERTa) without any specific task in mind.
  - It takes your tokenized text and processes it through all its layers.
  - Its primary output is the last_hidden_state. This is a rich, high-dimensional vector (an embedding) for every token in your input. This output represents a deep contextual understanding of the text.
  - You use AutoModel when you want to get these powerful embeddings to use for a custom task. For example, you might want to feed these embeddings into a different kind of machine learning model or build a completely new architecture on top of it. It gives you maximum flexibility.

Automodel vs AutoModelForSequenceClassification

- **AutoModel<task>**
  - This class loads the base model (AutoModel) and then adds a task-specific "head" on top of it. In this case, it adds a classification head.
  - It does everything AutoModel does, but then it takes the output of the base model (specifically, the embedding of the [CLS] token) and passes it through an additional small layer (a linear layer) that is designed for classification.
  - This model directly outputs the logits for your classification task. The size of this output matches the num_labels you specify.
  - You use this 99% of the time you are doing a standard text classification task. It's a complete, ready-to-use tool that's built for the job. The same logic applies to other classes like AutoModelForQuestionAnswering or AutoModelForTokenClassification

Automodel vs AutoModelForSequenceClassification

- So the core difference is what you get out of the model and how much work you have to do yourself.
- AutoModelForSequenceClassification is a Frozen Pizza: It's a complete, pre-assembled product designed for one specific task: making pizza. You just have to "heat it" (fine-tune it). It's incredibly convenient if all you want is pizza.
- AutoModel is just the Pizza Dough: It's the high-quality, essential base ingredient. You can't eat it as is. Its advantage is flexibility. With this dough, you can choose to make a pizza, or you could add your own toppings to make a calzone, garlic bread, or something entirely new. You have to build the final product yourself.